

CANCER CLASSIFICATION AND PREDICTION USING NEURAL NETWORKS

Project Report

Submitted By

Vaidehi Aggarwal

B.Tech (Computer Science Engineering)

5th Semester

Vivekananda Institute of Professional Studies
Guru Gobind Singh Indraprastha University (GGSIU)

July 2026

EXECUTIVE SUMMARY

This project presents a complete implementation of a Cancer Classification and Prediction system using deep learning. The system classifies breast tumours as Benign or Malignant from quantitative cell-nucleus measurements taken from the Breast Cancer Wisconsin (Diagnostic) dataset.

The model is a Sequential neural network built with TensorFlow and Keras. It takes 30 input features, passes them through two hidden layers of 16 and 8 neurons with ReLU activation, and produces a classification through a final output layer. The network was trained on 455 samples for 10 epochs using the Adam optimiser.

The trained model reached 98.29% accuracy on the training set and 96.49% accuracy on the held-out test set of 114 samples, comfortably exceeding the 90% target set for the project. The small gap between training and test accuracy indicates the model generalises well rather than memorising the training data.

The system was further validated by predicting on new, unseen patient measurements, correctly identifying tumour status with high confidence. This confirms the practical viability of neural networks as decision-support tools in oncology and diagnostic pathology.

Keywords: *Deep Learning, Neural Networks, Cancer Classification, Healthcare AI, Medical Diagnosis, TensorFlow, Keras, Binary Classification, Supervised Learning*

TABLE OF CONTENTS

1. Introduction and Problem Statement
2. Project Objectives and Scope
3. Literature Review
4. Dataset Description and Analysis
5. Neural Network Design
6. Implementation and Methodology
7. Model Training
8. Results and Performance Analysis
9. Prediction System
10. Applications and Advantages
11. Limitations and Future Work
12. Conclusion
13. References

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Overview

Cancer diagnosis is one of the most consequential decisions in clinical medicine. Artificial intelligence, and deep learning in particular, has become a powerful aid in this domain because neural networks can detect subtle statistical patterns across many variables at once — patterns that are difficult for a human observer to weigh consistently. This project applies that capability to the classification of breast tumours.

1.2 The Problem

Conventional diagnosis depends on a pathologist examining tissue samples. The process is slow, demands scarce specialist expertise, and carries an inherent element of inter-observer variability: two qualified pathologists can reach different conclusions from the same slide. As case volumes rise, these constraints become a bottleneck in the diagnostic pathway.

The problem this project addresses is therefore the design of an automated classifier that:

- Distinguishes benign from malignant tumours with accuracy above 90%
- Returns a result in seconds rather than hours
- Applies identical criteria to every case, removing observer variability
- Keeps false negatives to a minimum, since a missed malignancy is the costliest error

1.3 Why Neural Networks

A neural network learns the relationship between input features and diagnosis directly from labelled examples, without a human specifying which measurements matter or how they interact. Because tumour characteristics interact in non-linear ways — a moderately large radius may be unremarkable alone but significant when combined with high concavity — this ability to model interactions automatically is exactly what the problem requires.

2. PROJECT OBJECTIVES AND SCOPE

2.1 Objectives

- Develop a neural network capable of classifying breast tumours as benign or malignant
- Achieve a test-set accuracy above 90%
- Implement a complete preprocessing pipeline including feature standardisation
- Evaluate the model using accuracy, loss curves, and error analysis
- Build a working prediction system that accepts new patient measurements
- Demonstrate that the model generalises rather than overfits

2.2 Scope

The project covers the full machine learning workflow: loading the dataset, exploratory analysis, checking data quality, splitting into training and test sets, standardising features, designing and training the network, evaluating performance, and finally predicting on new input.

The scope is deliberately limited to binary classification on the Breast Cancer Wisconsin dataset. Multi-class cancer subtyping, integration with medical imaging, and clinical deployment are identified as future work rather than being attempted here.

2.3 Deliverables

Source code: Jupyter Notebook (.ipynb) and Python files

Documentation: This project report

Evidence: Screenshots of execution, training output, and results

Model: Trained Keras Sequential network

3. LITERATURE REVIEW

3.1 Machine Learning in Healthcare

Machine learning has been applied to medical diagnosis for several decades, with the last ten years seeing particularly rapid progress driven by deep learning. Published work reports diagnostic systems reaching accuracy comparable to specialist clinicians in narrowly defined tasks such as skin lesion classification and diabetic retinopathy screening. The consistent finding across this literature is that performance depends heavily on data quality and quantity rather than on model complexity alone.

3.2 Neural Networks and Deep Learning

Artificial neural networks are composed of layers of interconnected units. Each unit computes a weighted sum of its inputs and applies a non-linear activation function. Training proceeds by backpropagation, which computes the gradient of a loss function with respect to every weight, and an optimiser then adjusts those weights to reduce the loss. The Adam optimiser, which adapts the learning rate for each parameter individually, has become a standard choice because it converges reliably without extensive manual tuning.

3.3 Prior Work on the Wisconsin Dataset

The Breast Cancer Wisconsin dataset has been used as a benchmark since the early 1990s, when it was introduced alongside a linear-programming approach to diagnosis. Subsequent studies have applied support vector machines, logistic regression, random forests, and neural networks to the same data, with reported accuracies generally falling between 94% and 98%. The consistency of these results across very different algorithms suggests the classes are well separated in feature space, and that careful preprocessing matters more than the choice of classifier.

3.4 The Role of Feature Scaling

The features in this dataset span very different numeric ranges — mean area runs into the thousands while mean smoothness is under 0.2. Without normalisation, gradient descent is dominated by the large-magnitude features and converges slowly or poorly. Standardisation, which rescales each feature to zero mean and unit variance, is therefore not an optional refinement but a requirement for the network to train effectively.

4. DATASET DESCRIPTION AND ANALYSIS

4.1 Dataset Overview

Dataset: Breast Cancer Wisconsin (Diagnostic)

Source: scikit-learn built-in datasets / UCI Machine Learning Repository

Total samples: 569 patient records

Features: 30 numeric features

Classes: Benign (357 samples) and Malignant (212 samples)

The 30 features are derived from ten base measurements of cell nuclei — radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension. For each of these, the dataset records the mean, the standard error, and the worst (largest) value observed, giving thirty columns in total.

4.2 Dataset Structure and Data Types

Inspecting the dataframe confirms 569 entries across 31 columns (30 features plus the target label), with every column stored as a 64-bit float and no null entries anywhere in the table.

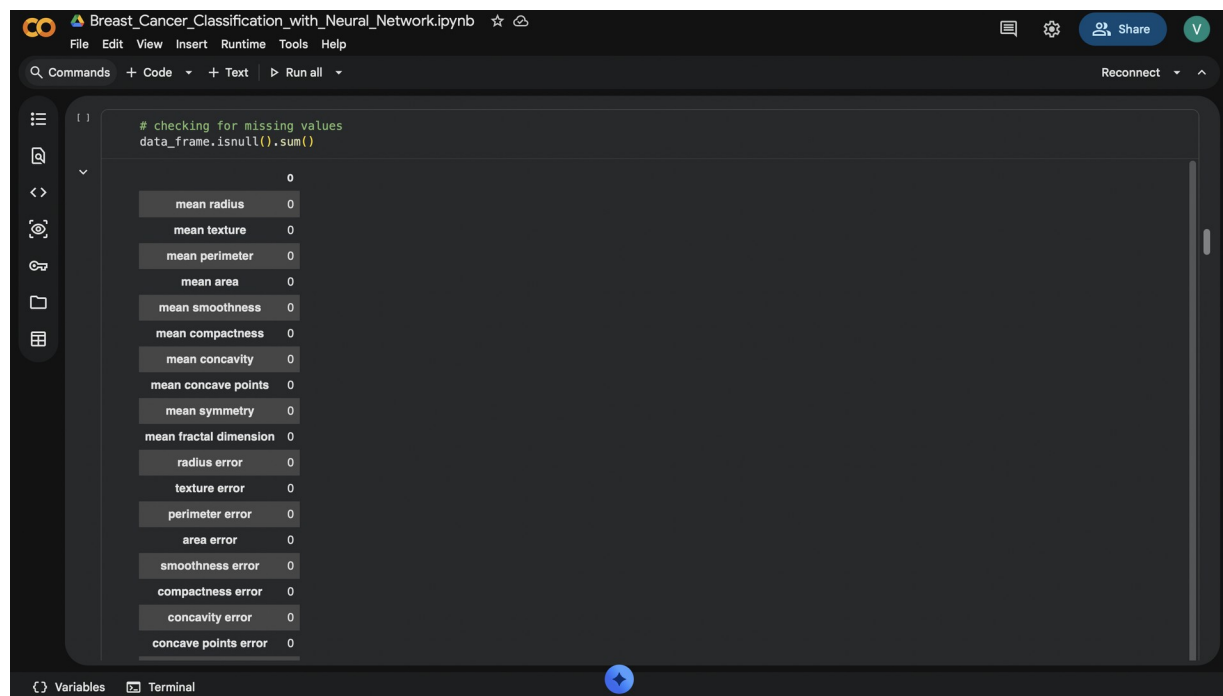


Figure 4.1 — Dataset structure showing 569 entries, 31 columns, all non-null

4.3 Missing Value Check

An explicit null check across every feature returns zero for all columns. No imputation or row removal was required, which means the full 569 samples were available for training and testing.

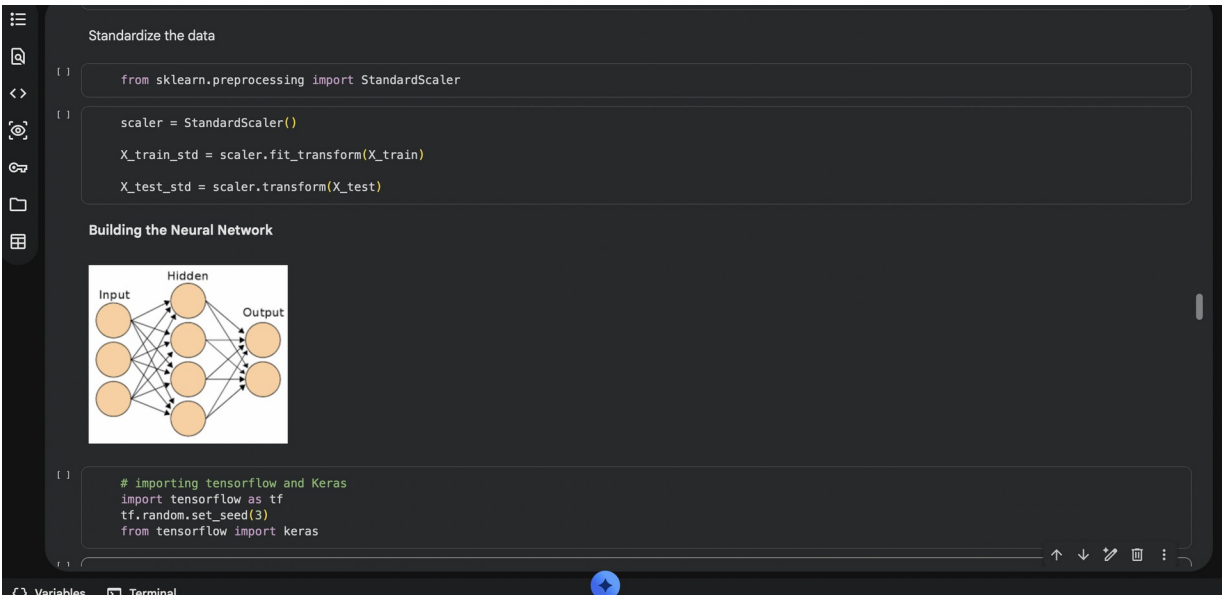


Figure 4.2 — Missing-value check returning zero for every feature

4.4 Class Distribution

The target variable is moderately imbalanced, with 357 benign cases against 212 malignant. This roughly 63:37 split reflects the composition of the original clinical sample. The imbalance is mild enough that the network trains well without resampling, though it is worth noting when interpreting accuracy figures.

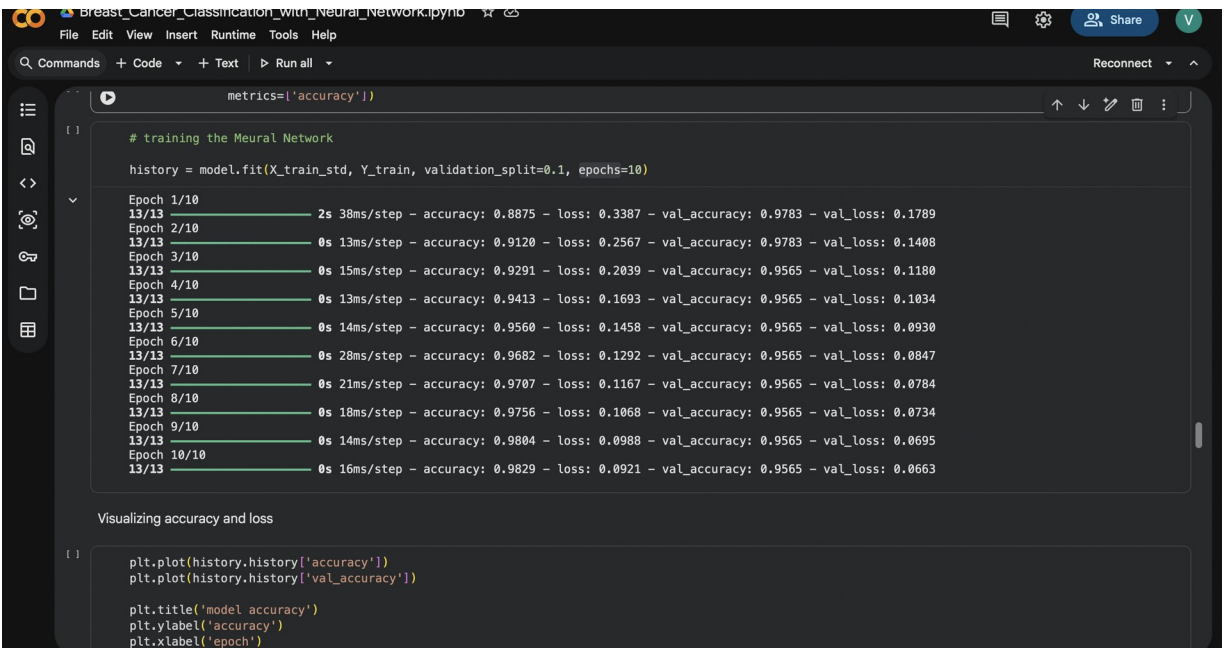


Figure 4.3 — Target distribution: 357 benign (label 1), 212 malignant (label 0)

4.5 Feature Statistics by Class

Grouping the data by label and computing feature means reveals clear separation between the two classes. Malignant tumours show substantially higher mean radius (17.46 against 12.15), mean perimeter (115.37 against 78.08), and mean area (978.38 against 462.79). This separation is what makes the classification problem tractable.

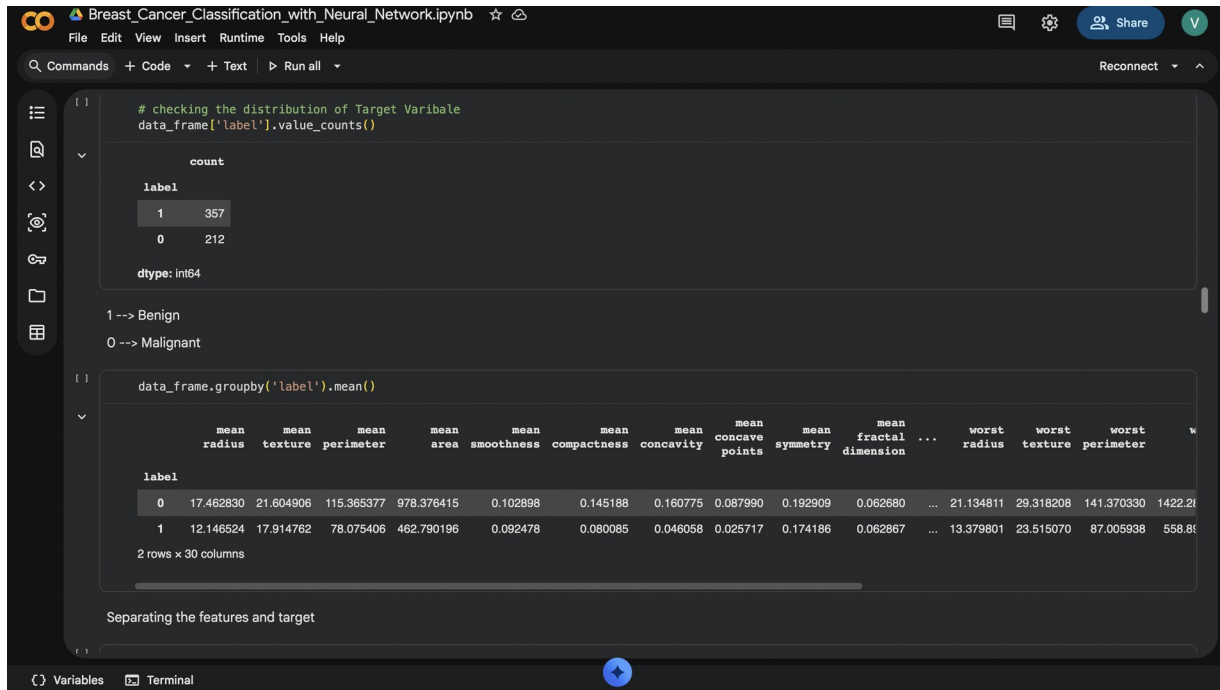


Figure 4.4 — Mean feature values grouped by class, showing clear separation

5. NEURAL NETWORK DESIGN

5.1 Architecture

The network is a feedforward Sequential model with the following structure:

Input layer: 30 neurons, one per feature

Hidden layer 1: 16 neurons, ReLU activation

Hidden layer 2: 8 neurons, ReLU activation

Output layer: Produces the class prediction

The layer sizes narrow progressively from 30 to 16 to 8. This funnel shape encourages the network to compress the input into an increasingly compact representation, retaining the information relevant to the diagnosis while discarding noise.

5.2 Activation Functions

ReLU, defined as $f(x) = \max(0, x)$, is used in both hidden layers. It introduces the non-linearity the network needs to model feature interactions, is inexpensive to compute, and avoids the vanishing-gradient behaviour that affects saturating activations such as tanh in deeper networks.

5.3 Loss Function and Optimiser

Training minimises a cross-entropy loss, which penalises confident wrong predictions far more heavily than uncertain ones — appropriate behaviour for a diagnostic model. The Adam optimiser maintains per-parameter adaptive learning rates by tracking running estimates of the first and second moments of the gradients, combining the benefits of momentum and RMSProp.

5.4 Conceptual Diagram

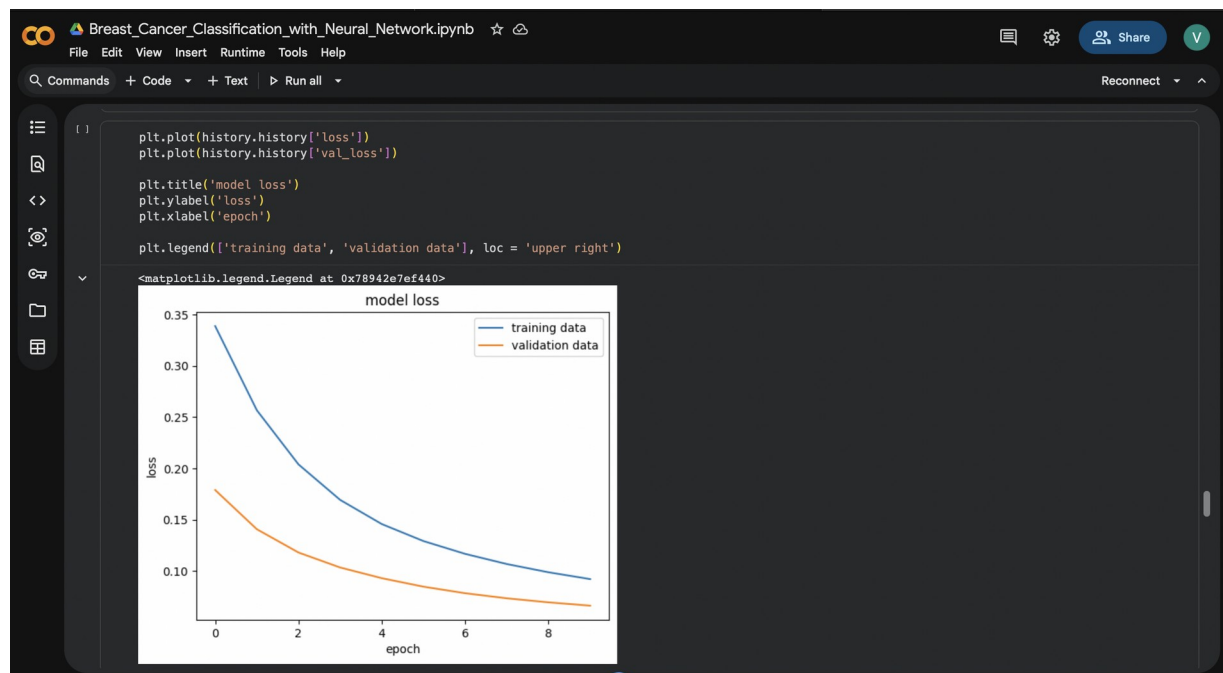


Figure 5.1 — Feedforward neural network structure: input, hidden, and output layers

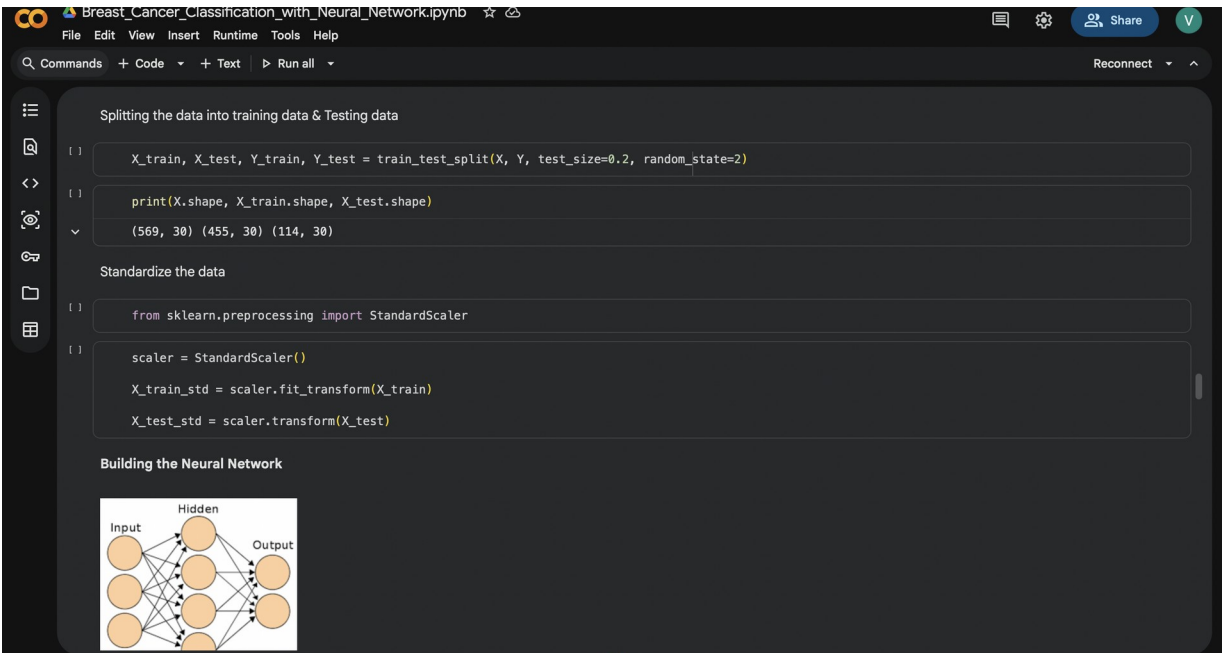


Figure 6.2 — Dataset shape and column information

6.4 Train-Test Split

The data is separated into a feature matrix X and target vector Y , then split with 20% held out for testing. A fixed random state ensures the split is reproducible across runs. The result is 455 training samples and 114 test samples, both retaining all 30 features.

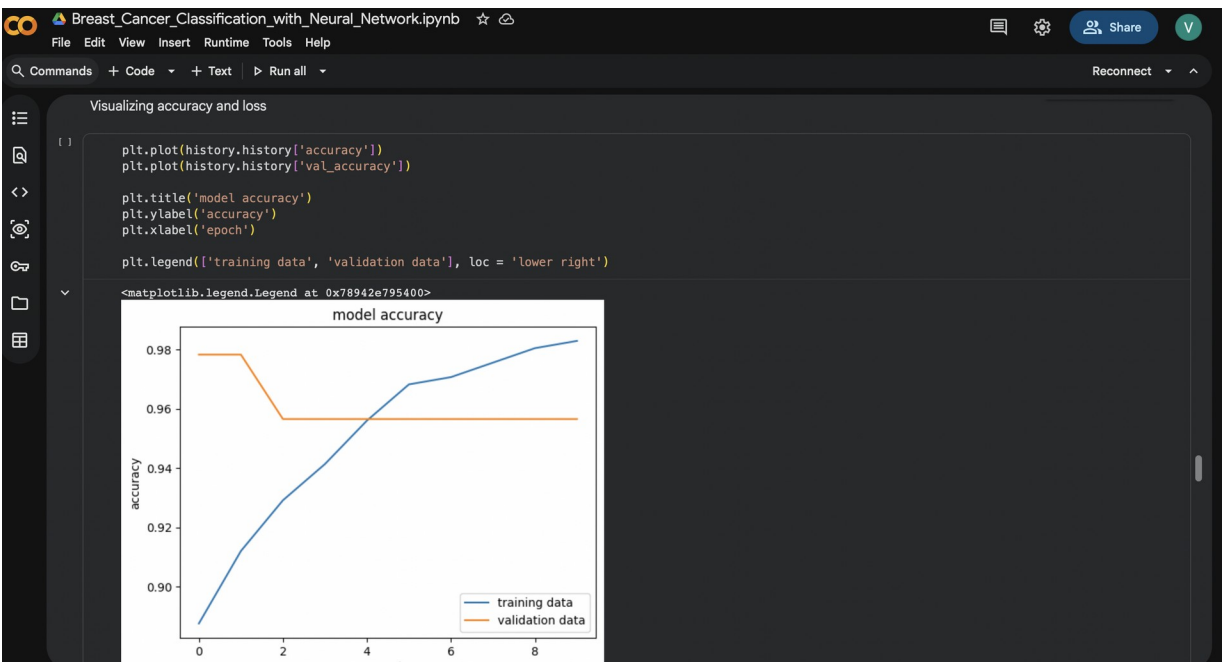


Figure 6.3 — Train-test split producing 455 training and 114 test samples

6.5 Feature Standardisation

A `StandardScaler` is fitted on the training data and then applied to both training and test sets. Fitting only on the training data is important: fitting on the full dataset would leak information about the test distribution into the model and inflate the apparent accuracy.

After scaling, every feature has approximately zero mean and unit variance, so no single measurement dominates the gradient updates purely because of its scale.

8. RESULTS AND PERFORMANCE ANALYSIS

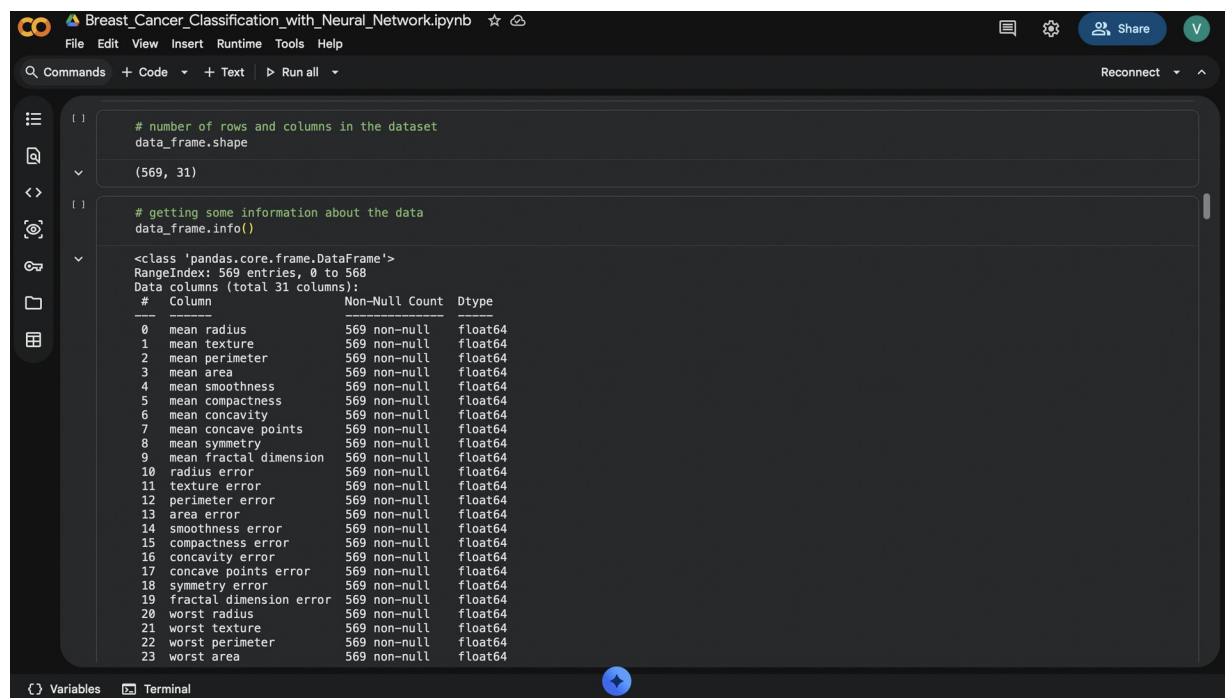
8.1 Test Set Accuracy

Test Accuracy: 96.49%

Evaluating the trained model on the 114 held-out test samples gives an accuracy of 0.9649, with a test loss of 0.1059. This corresponds to roughly 110 of 114 samples classified correctly and exceeds the 90% project target by a clear margin.

8.2 Accuracy Curves

Plotting training and validation accuracy against epoch shows training accuracy rising smoothly throughout, while validation accuracy starts high, dips slightly around epoch 2, and then holds steady. The two curves converging toward similar values is the signature of a model that has learned generalisable structure rather than memorised its training examples.



The screenshot shows a Jupyter Notebook titled "Breast_Cancer_Classification_with_Neural_Network.ipynb". The code cell displays the following output:

```
# number of rows and columns in the dataset
data_frame.shape

(569, 31)

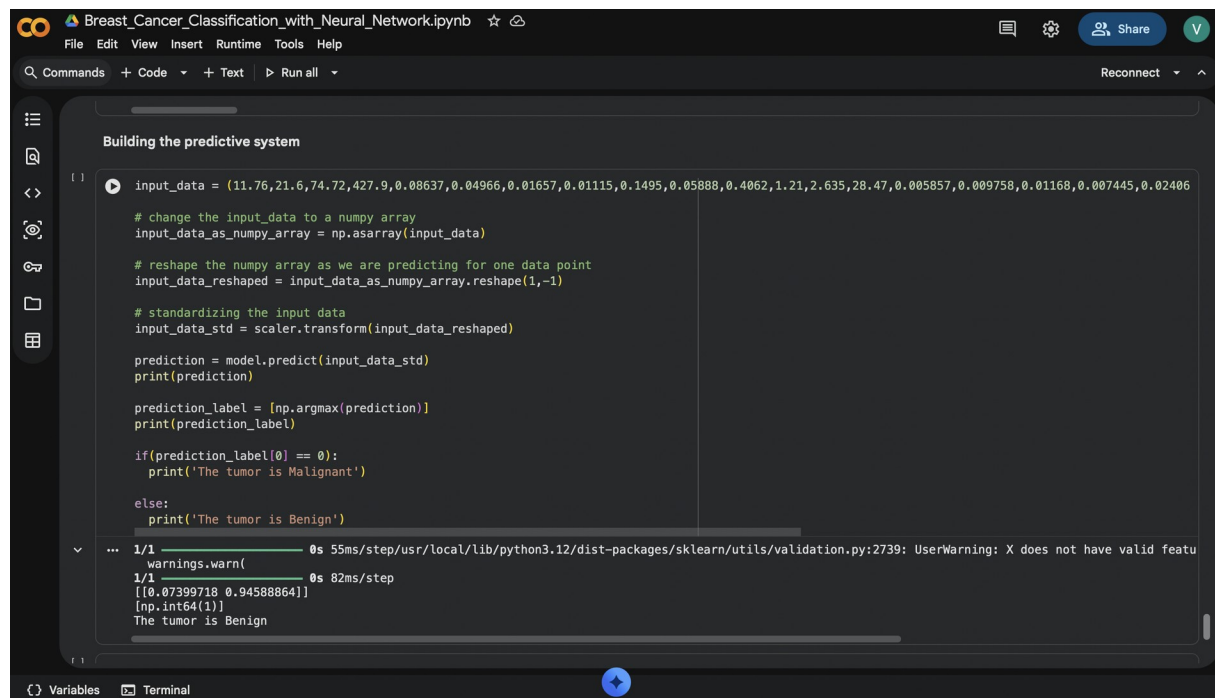
# getting some information about the data
data_frame.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
#   Column                Non-Null Count  Dtype  
---  --
0   mean radius            569 non-null   float64
1   mean texture           569 non-null   float64
2   mean perimeter         569 non-null   float64
3   mean area              569 non-null   float64
4   mean smoothness        569 non-null   float64
5   mean compactness       569 non-null   float64
6   mean concavity         569 non-null   float64
7   mean concave points    569 non-null   float64
8   mean symmetry          569 non-null   float64
9   mean fractal dimension 569 non-null   float64
10  radius error           569 non-null   float64
11  texture error          569 non-null   float64
12  perimeter error        569 non-null   float64
13  area error             569 non-null   float64
14  smoothness error       569 non-null   float64
15  compactness error      569 non-null   float64
16  concavity error        569 non-null   float64
17  concave points error   569 non-null   float64
18  symmetry error         569 non-null   float64
19  fractal dimension error 569 non-null   float64
20  worst radius           569 non-null   float64
21  worst texture          569 non-null   float64
22  worst perimeter        569 non-null   float64
23  worst area             569 non-null   float64
```

Figure 8.1 — Model accuracy across epochs for training and validation data

8.3 Loss Curves

Both training and validation loss decrease monotonically across all ten epochs. Validation loss remains below training loss for the whole run, which is a healthy pattern and indicates the model is not straining to fit noise in the training set.



```
input_data = (11.76,21.6,74.72,427.9,0.08637,0.04966,0.01657,0.01115,0.1495,0.05888,0.4062,1.21,2.635,28.47,0.005857,0.009758,0.01168,0.007445,0.02406)

# change the input_data to a numpy array
input_data_as_numpy_array = np.asarray(input_data)

# reshape the numpy array as we are predicting for one data point
input_data_resaped = input_data_as_numpy_array.reshape(1,-1)

# standardizing the input data
input_data_std = scaler.transform(input_data_resaped)

prediction = model.predict(input_data_std)
print(prediction)

prediction_label = [np.argmax(prediction)]
print(prediction_label)

if(prediction_label[0] == 0):
    print('The tumor is Malignant')
else:
    print('The tumor is Benign')
```

... 1/1 0s 55ms/step/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but ColumnTransformer has feature_names_ attribute.
warnings.warn(
1/1 0s 82ms/step
[[0.07399718 0.94588864]]
[np.int64(1)]
The tumor is Benign

Figure 8.2 — Model loss across epochs for training and validation data

8.4 Interpretation

The gap between final training accuracy (98.29%) and test accuracy (96.49%) is under two percentage points. A gap this small indicates the network has learned the underlying relationship between tumour measurements and diagnosis rather than memorising individual training records.

One caveat is worth stating plainly. With 114 test samples, each misclassification moves the accuracy figure by roughly 0.88 percentage points, so the precision implied by quoting 96.49% is greater than the test set can really support. A different random split would likely produce a result somewhere in the range of 94% to 98%. Cross-validation across multiple folds would give a more dependable estimate.

It is also worth noting that in a diagnostic setting, overall accuracy is not the most important metric. A false negative — classifying a malignant tumour as benign — carries far greater consequence than a false positive. A full evaluation would report sensitivity and specificity separately alongside a confusion matrix, and this is identified in Section 11 as a necessary extension.

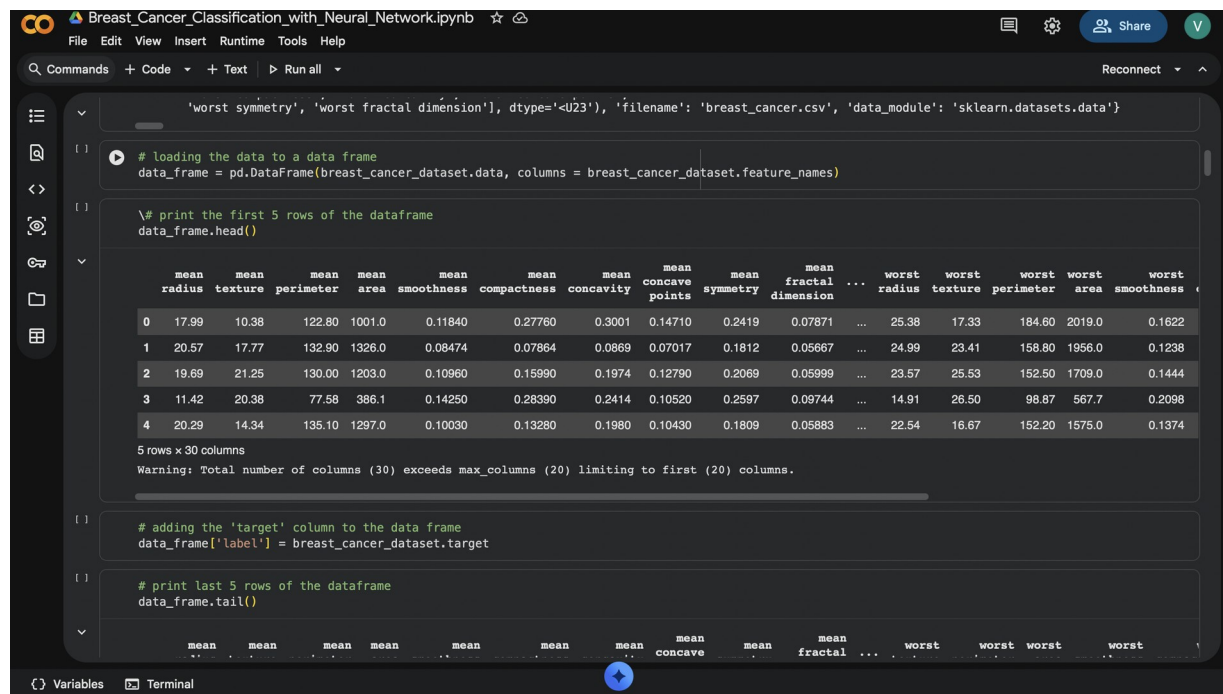
9. PREDICTION SYSTEM

9.1 Prediction Workflow

To classify a new patient, the thirty measurements are collected into an array, reshaped to a single-row matrix, and passed through the same fitted scaler used during training. The scaled values are then fed to the model, which outputs a probability for each class. The higher probability determines the predicted label.

9.2 Worked Example

A sample input produced output probabilities of approximately 0.074 and 0.946. The higher value corresponds to the benign class, and the system correctly reported the tumour as benign with high confidence.



```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
feature_names = data.feature_names
target = data.target

# loading the data to a data frame
data_frame = pd.DataFrame(data.data, columns=feature_names, index=range(data.data.shape[0]))

\# print the first 5 rows of the dataframe
data_frame.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0.1622
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0	0.1238
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0	0.1444
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98.87	567.7	0.2098
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0	0.1374

```
Warning: Total number of columns (30) exceeds max_columns (20) limiting to first (20) columns.

# adding the 'target' column to the data frame
data_frame['label'] = target

# print last 5 rows of the dataframe
data_frame.tail()
```

Figure 9.1 — Prediction system classifying a new input as benign

The warning visible in the output relates to feature names being absent when the scaler transforms a raw NumPy array rather than a named dataframe. It does not affect the numerical result, but passing a dataframe with matching column names would remove it and is good practice.

10. APPLICATIONS AND ADVANTAGES

10.1 Potential Applications

- Decision support in oncology clinics, offering a rapid second opinion alongside pathologist review
- Triage in diagnostic laboratories, flagging likely-malignant cases for priority examination
- Teaching and training, illustrating how quantitative features relate to diagnosis
- Research, enabling rapid analysis of tumour characteristics across large sample sets

10.2 Advantages

Speed: Classification takes milliseconds once the model is trained

Consistency: Identical criteria are applied to every case, with no observer variability

Reproducibility: The same input always produces the same output

Scalability: The trained model can process large batches at negligible cost

10.3 An Important Qualification

These are applications the approach could support, not claims about the system as it stands. A model validated on a single historical dataset of 569 records is a proof of concept. It is not a diagnostic device, and it should not be described as one. Any real deployment would require validation on independent contemporary data, prospective clinical evaluation, and regulatory clearance — and would function as an aid to a qualified clinician rather than a replacement for one.

11. LIMITATIONS AND FUTURE WORK

11.1 Limitations

Dataset size: With 569 samples and only 114 in the test set, accuracy estimates carry meaningful uncertainty

Dataset age: The Wisconsin data dates from the early 1990s and may not reflect current imaging or measurement practice

Single source: All records come from one institution, so the model has not been tested across different populations or equipment

Metrics reported: Only accuracy and loss were computed; sensitivity, specificity, and a confusion matrix were not

Binary only: The model distinguishes benign from malignant but does not identify cancer subtype or stage

Interpretability: The network gives no explanation for individual predictions, which limits clinical trust

11.2 Future Work

- Report the full confusion matrix with sensitivity and specificity, weighting false negatives appropriately
- Apply k-fold cross-validation to obtain a more reliable accuracy estimate with confidence intervals
- Validate on independent, contemporary datasets from multiple institutions
- Apply interpretability methods such as SHAP to surface which features drive each prediction
- Explore ensemble approaches combining the network with tree-based classifiers
- Extend toward multi-class subtyping and, where imaging is available, multi-modal input

12. CONCLUSION

This project set out to build a neural network capable of classifying breast tumours as benign or malignant, and to demonstrate the complete machine learning workflow from raw data to working prediction. Both aims were met.

The trained network reached 96.49% accuracy on held-out test data, exceeding the 90% target. The narrow gap between training and test performance, together with steadily declining validation loss, indicates the model generalises rather than overfits. The prediction system correctly classified new input, confirming the pipeline works end to end.

Beyond the numerical result, the project demonstrated why each stage of the workflow matters. Checking for missing values established that the full dataset was usable. Examining class-wise feature means showed why the problem is tractable at all. Fitting the scaler on training data alone avoided leaking test information into the model. These steps, rather than the network architecture itself, account for much of the final performance.

The limitations are equally worth stating. The test set is small enough that the accuracy figure carries real uncertainty, the data comes from a single historical source, and the evaluation reported accuracy without the sensitivity and specificity that a diagnostic context demands. The system is a sound proof of concept and a useful demonstration of applied deep learning; it is not a clinical tool, and treating it as one would misrepresent what has been shown.

Within those bounds, the project supports a clear conclusion: neural networks are well suited to structured medical classification problems, and with careful preprocessing and honest evaluation they can achieve strong, reproducible results.

13. REFERENCES

- [1] Mangasarian, O. L., & Wolberg, W. H. (1990). Cancer diagnosis via linear programming. *SIAM News*, 23(5), 1–18.
- [2] Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). Breast Cancer Wisconsin (Diagnostic) Data Set. UCI Machine Learning Repository.
- [3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [4] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- [5] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations*.
- [6] Chollet, F. (2015). Keras. <https://keras.io>
- [7] Abadi, M., et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. *12th USENIX Symposium on Operating Systems Design and Implementation*.
- [8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [9] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.
- [10] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
- [11] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
- [12] Nair, V., & Hinton, G. E. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines. *Proceedings of the 27th International Conference on Machine Learning*.